

大模型技术及开发

基础篇：深度学习基础

主讲人：陈小军

时间：2025.3.14

目录

contents

01 | 深度学习简介

02 | PyTorch

01

深度学习简介

202X

有监督学习

定义：通过范例来训练在机器上运行的模型，而非显示编程。当模型输出错误时，调整模型的参数。

语音 → 文字

图像 → 类别

文章 → 主题

英文 → 主中文

人脸 → 姓名



▶ 传统机器学习范式



手工特征提取：人工设计算法在从原始数据中提取出具有判别性的特征，以便后续的分类、识别或检测任务。例如，在图像处理领域，SIFT (尺度不变特征变换)、HOG (方向梯度直方图) 和 LBP (局部二值模式)。

特征提取

训练分类器

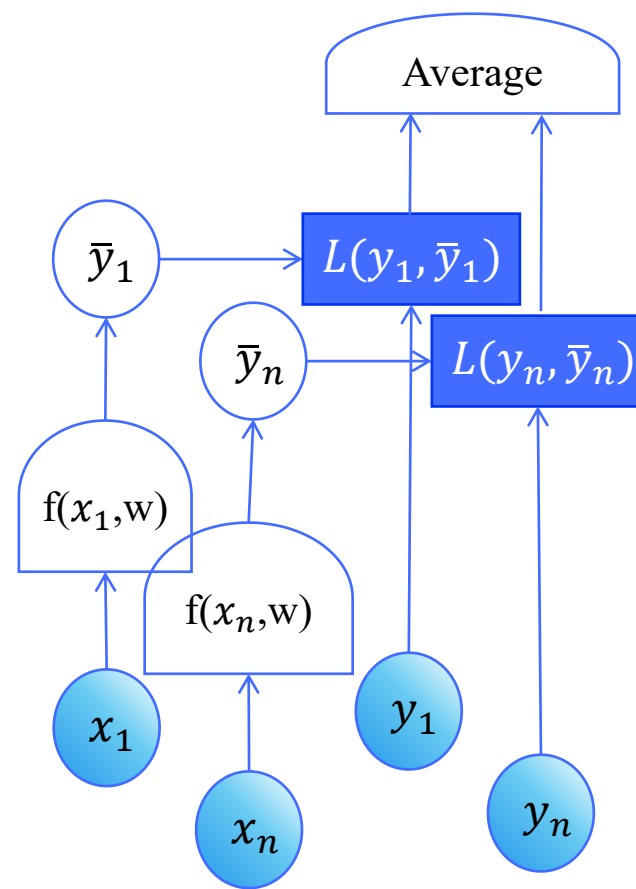


传统分类器：模型结构简单，对特征质量敏感，通常只针对任务有效，难以应用到其他任务。



▶ 参数模型

- $X = \{x_1, \dots, x_n\}$: 输入样本
- $\bar{y} = f(x_i, W)$: 参数模型, 如:
 - 线性回归模型 $f(x_i, W) = \sum_{j=1}^m x_{ij} w_j$
 - 线性回归模型 $f(x_i, W) = \text{softmax}(x_{i1} w_{11}, \dots, x_{i,m} w_{m,c})$
 - k 近邻模型 $f(x_i, W) = \underset{k}{\operatorname{argmax}} \|x_i - w_k\|^2$
 - SVM, 决策树, 神经网络...
- $L(y, \bar{y})$ 为损失函数, 如
 - MSE: $L(y_i, \bar{y}_i) = \|y_i - \bar{y}_i\|^2$
 - Cross entropy: $L(y_i, \bar{y}_i) = \frac{1}{c} \sum_{l=1}^c y_{il} \log \bar{y}_{il}$





线性模型示例

```
import numpy as np
import matplotlib.pyplot as plt

x_data = [1.0, 2.0, 3.0]
y_data = [2.0, 4.0, 6.0]

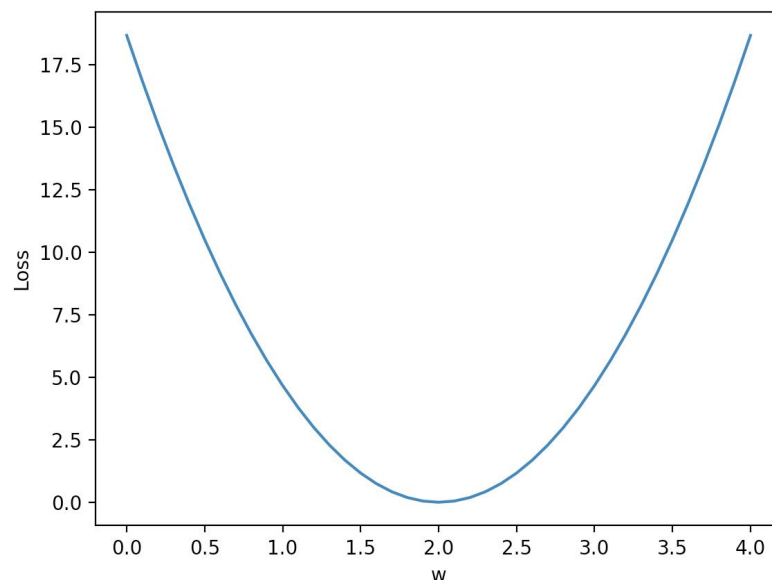
# our model forward pass
def forward(x):
    return x*w

# Loss function
def loss(x, y):
    y_pred = forward(x)
    return (y_pred-y)*(y_pred-y)

w_list = []
mse_list = []

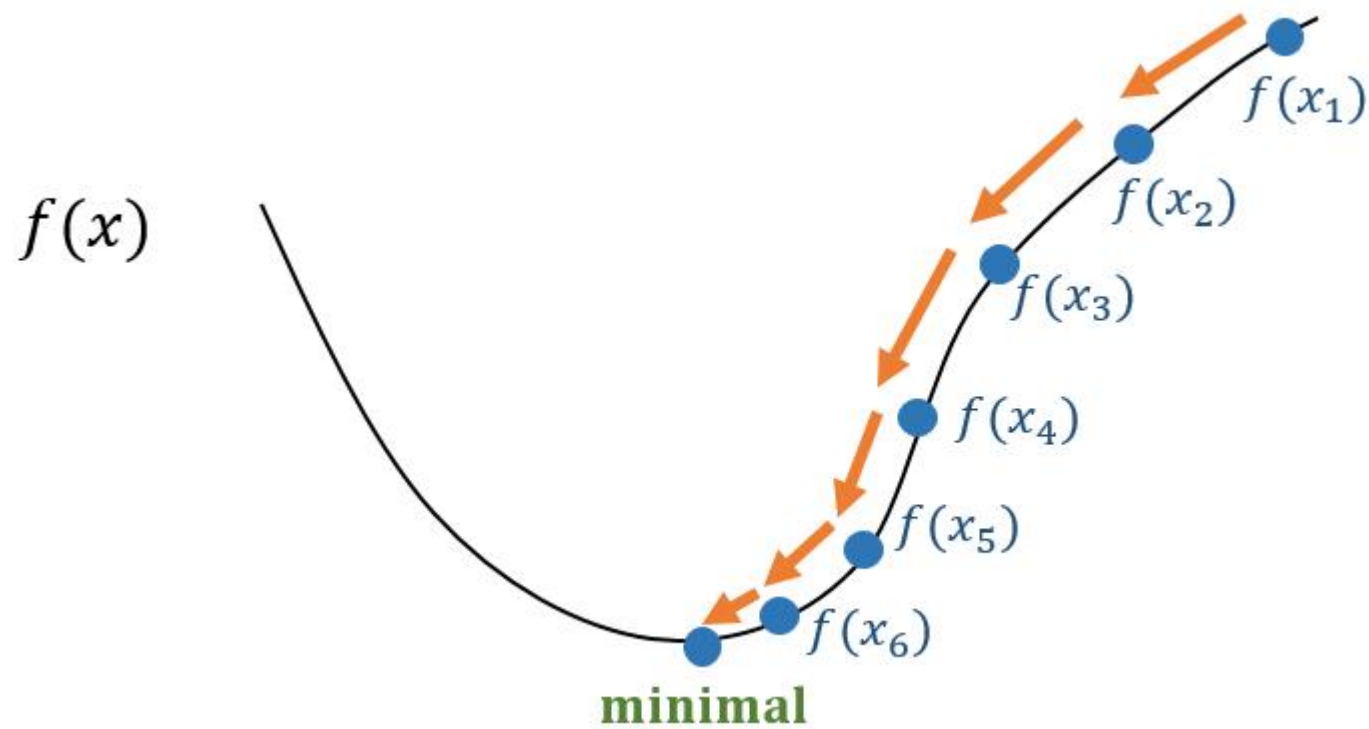
for w in np.arange(0.0, 4.1, 0.1):
    print("w=", w)
    l_sum = 0
    for x_val, y_val in zip(x_data, y_data):
        y_pred = forward(x_val)
        l = loss(x_val, y_val)
        l_sum += l
        print("\t", x_val, y_val, y_pred, l)
    print("NSE=", l_sum/3)
    w_list.append(w)
    mse_list.append(l_sum/3)

plt.plot(w_list, mse_list)
plt.ylabel('Loss')
plt.xlabel('w')
plt.show()
```



```
w= 0.0
    1.0 2.0 0.0 4.0
    2.0 4.0 0.0 16.0
    3.0 6.0 0.0 36.0
NSE= 18.6666666667
w= 0.1
    1.0 2.0 0.1 3.61
    2.0 4.0 0.2 14.44
    3.0 6.0 0.3 32.49
NSE= 16.8466666667
w= 0.2
    1.0 2.0 0.2 3.24
    2.0 4.0 0.4 12.96
    3.0 6.0 0.6 29.16
NSE= 15.12
w= 0.3
    1.0 2.0 0.3 2.89
    2.0 4.0 0.6 11.56
    3.0 6.0 0.9 26.01
NSE= 13.4866666667
w= 0.4
    1.0 2.0 0.4 2.56
    2.0 4.0 0.8 10.24
    3.0 6.0 1.2 23.04
NSE= 11.9466666667
w= 0.5
    1.0 2.0 0.5 2.25
    2.0 4.0 1.0 9.0
    3.0 6.0 1.5 20.25
NSE= 10.5
```


▶ 梯度下降



$$w = w - \lambda \frac{\partial L}{\partial w}$$



模型训练

```

x_data = [1.0, 2.0, 3.0]
y_data = [2.0, 4.0, 6.0]

w = 1.0 # any random value

# our model forward pass
def forward(x):
    return x*w

# Loss function
def loss(x, y):
    y_pred = forward(x)
    return (y_pred-y)*(y_pred-y)

# compute gradient
def gradient(x, y): # d_loss/d_w
    return 2*x*(x*w-y)

# Before training
print("predict (before training)", 4, forward(4))

# Training loop
for epoch in range(10):
    for x, y in zip(x_data, y_data):
        grad = gradient(x, y)
        w = w - 0.01 * grad
        print("\tgrad: ", x, y, grad)
        l = loss(x, y)

    print("progress:", epoch, l)

# After training
print("predict (after training)", 4, forward(4))

```

```

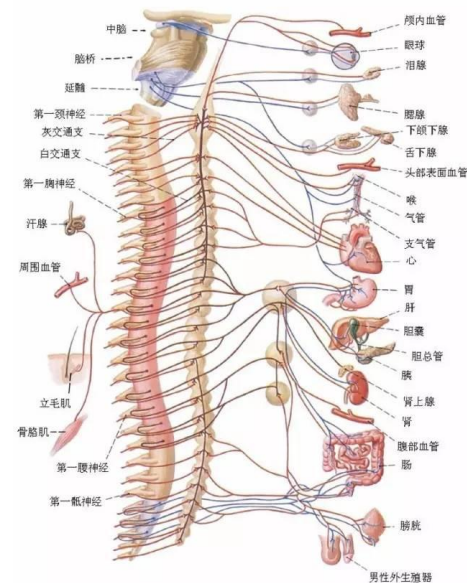
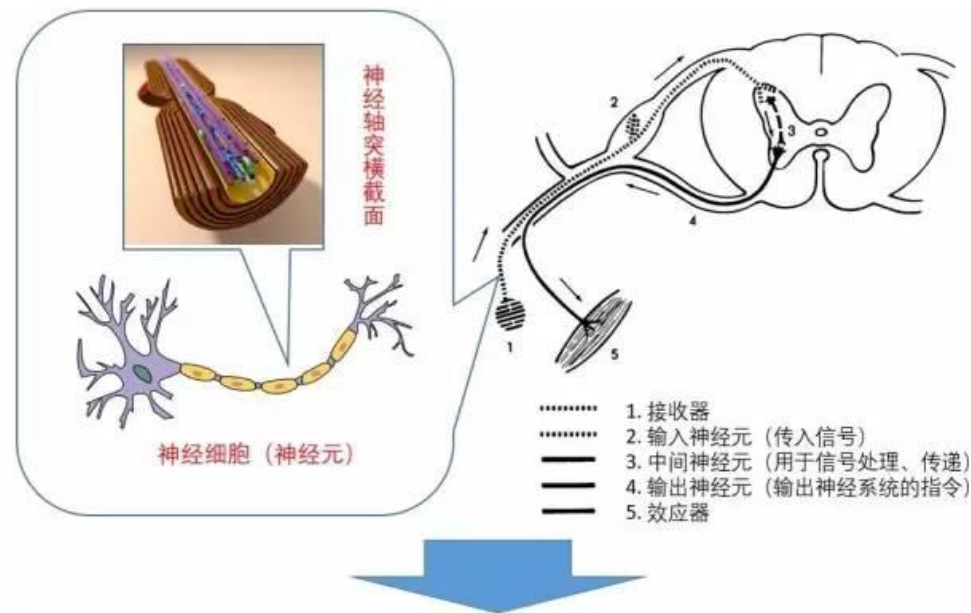
predict (before training) 4 4.0
grad: 1.0 2.0 -2.0
grad: 2.0 4.0 -7.84
grad: 3.0 6.0 -16.2288
progress: 0 4.919240100095999
grad: 1.0 2.0 -1.478624
grad: 2.0 4.0 -5.796206079999999
grad: 3.0 6.0 -11.998146585599997
progress: 1 2.688769240265834
grad: 1.0 2.0 -1.093164466688
grad: 2.0 4.0 -4.285204709416961
grad: 3.0 6.0 -8.87037374849311
progress: 2 1.4696334962911515
grad: 1.0 2.0 -0.8081896081960389
grad: 2.0 4.0 -3.1681032641284723
grad: 3.0 6.0 -6.557973756745939
progress: 3 0.8032755585999681
grad: 1.0 2.0 -0.59750427561463
grad: 2.0 4.0 -2.3422167604093502
grad: 3.0 6.0 -4.848388694047353
progress: 4 0.43905614881022015
grad: 1.0 2.0 -0.44174208101320334
grad: 2.0 4.0 -1.7316289575717576
grad: 3.0 6.0 -3.584471942173538
progress: 5 0.2399802903801062
grad: 1.0 2.0 -0.3265852213980338
grad: 2.0 4.0 -1.2802140678802925
grad: 3.0 6.0 -2.650043120512205
progress: 6 0.1311689630744999
grad: 1.0 2.0 -0.241448373202223
grad: 2.0 4.0 -0.946477622952715
grad: 3.0 6.0 -1.9592086795121197
progress: 7 0.07169462478267678
grad: 1.0 2.0 -0.17850567968888198
grad: 2.0 4.0 -0.6997422643804168
grad: 3.0 6.0 -1.4484664872674653
progress: 8 0.03918700813247573
grad: 1.0 2.0 -0.13197139106214673
grad: 2.0 4.0 -0.5173278529636143
grad: 3.0 6.0 -1.0708686556346834
progress: 9 0.021418922423117836
predict (after training) 4 7.804863933862125

```

▶ 大脑及神经系统

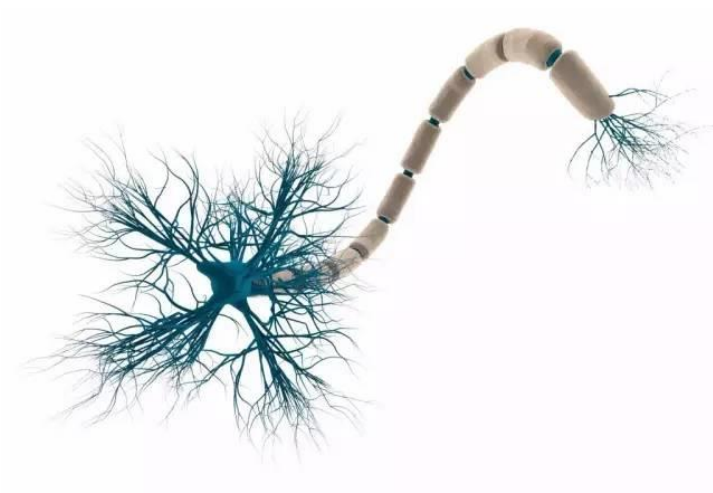


大脑

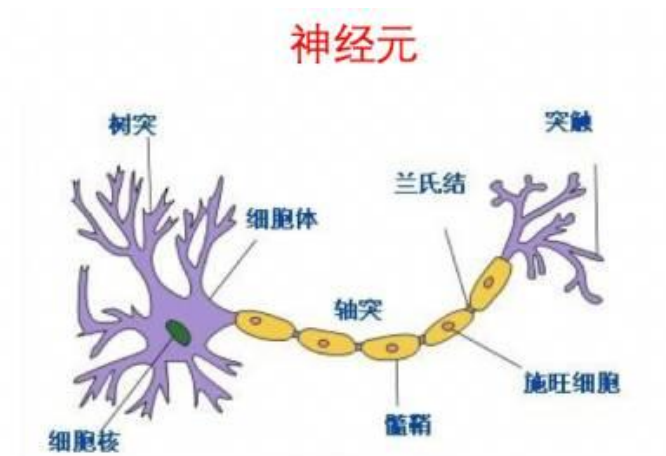


自主神经系概观

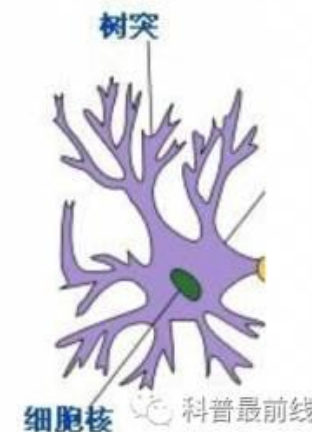
▶ 神经元



神经元



神经胶质细胞



神经元具体结构

树突：有多个，主要用来接受传入信息

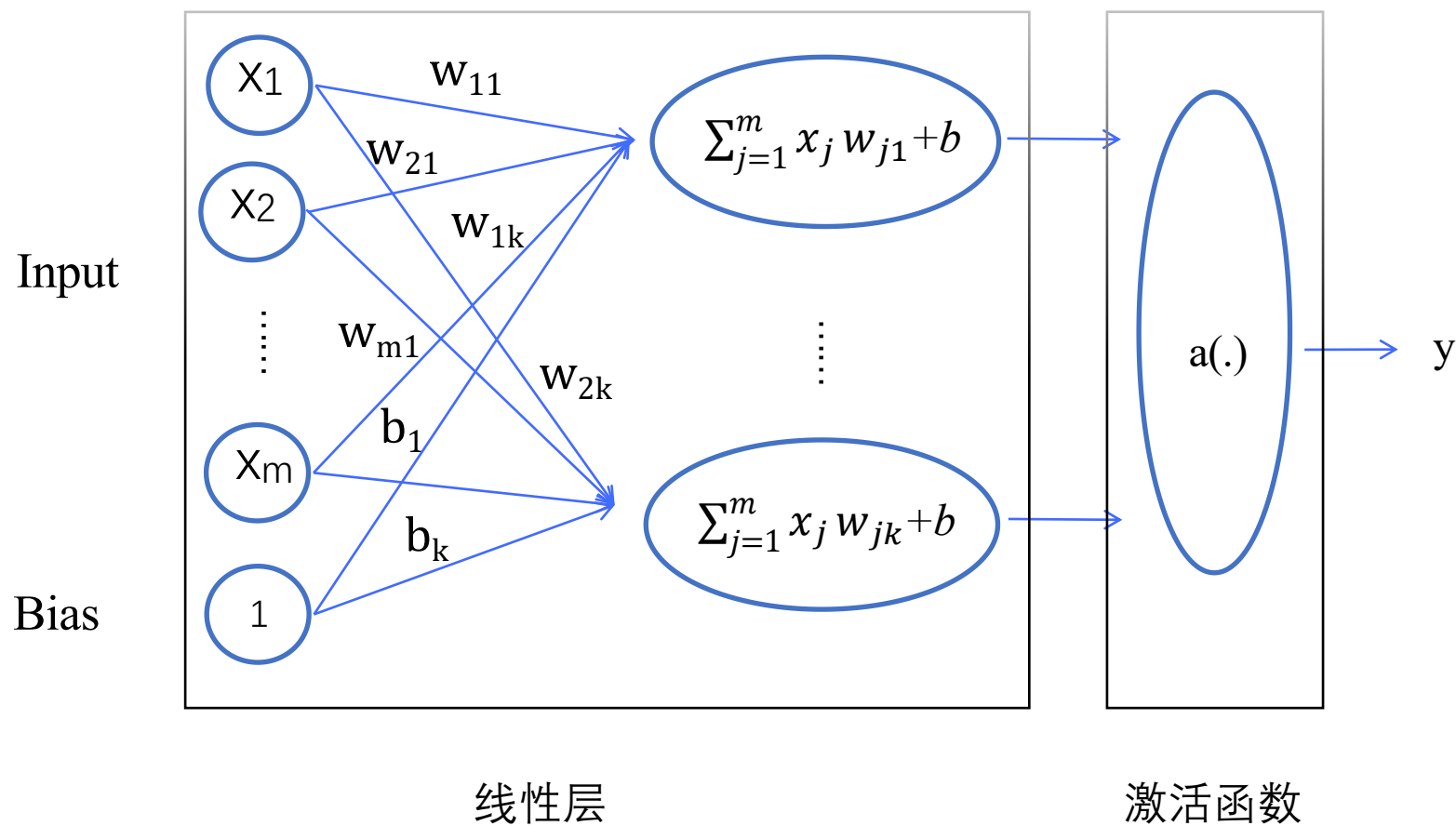
细胞核：对信号进行计算

轴突：只有一条，用于信号传递

突触：轴突末梢跟其他神经元的树突产生连接，从而传递信号。这个连接的位置在生物学上叫做“突触”。

一个神经元接入了多个输入，经过计算最终只变成一个输出，传递给了后面的神经元

人工神经网络



多层感知器代码示例

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class ExampleNet(nn.Module):
    def __init__(self, input_size, output_size):
        super(ExampleNet, self).__init__()
        self.linear = nn.Linear(input_size, output_size)

    def forward(self, x):
        x = self.linear(x)
        x = F.relu(x)
        x = F.softmax(x, dim=1) # dim=1 for applying softmax along rows
        return x

# Example Usage:
input_size, output_size, batch_size = 10, 5, 2

# Create an instance of the network
net = ExampleNet(input_size, output_size)

# Generate a random input tensor
input_tensor = torch.randn(batch_size, input_size)

# Pass the input through the network
output_tensor = net(input_tensor)

# Print the output
print("Input Tensor:")
print(input_tensor)
print("\nOutput Tensor (after Linear, ReLU, and Softmax):")
print(output_tensor)
```

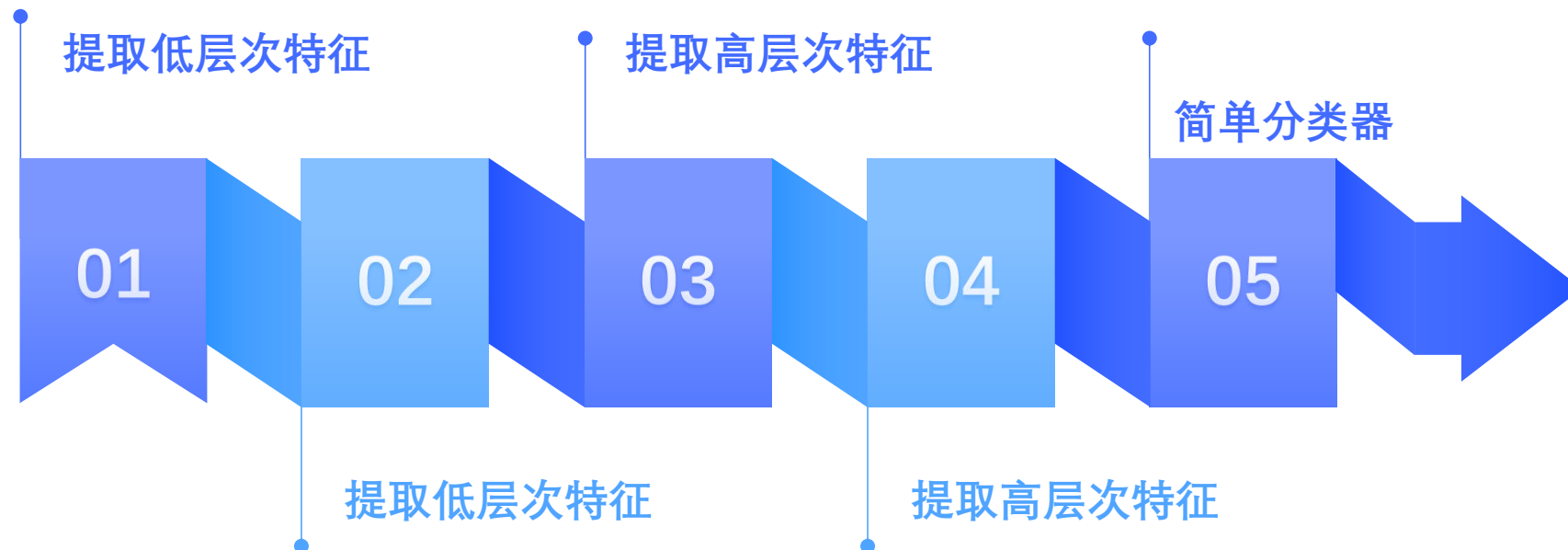
1 设计模型



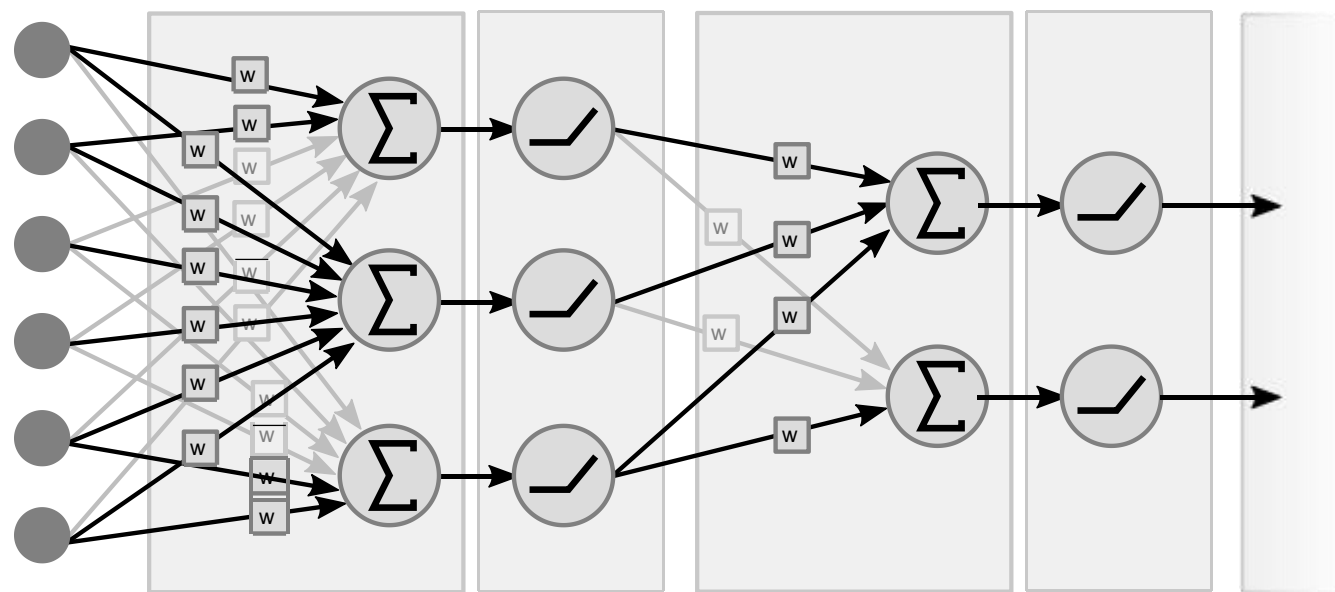
2 构造损失函数及优化器

3 开始训练(前向计算forward(), 反向传播计算梯度backward(), 更新step())

深度学习-多层神经网络



深度神经网络



深度学习-多层神经网络

- 线性块: $s_{k+1} = w_k z_k$
- 非线性块: $z_{k+1} = h(s_{k+1})$

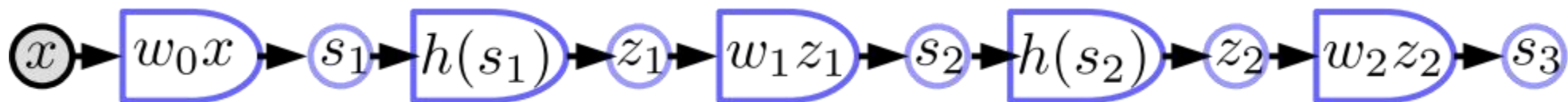
使用PyTorch:

- `nn.Linear` class
- `torch.relu` function
- State variables are temporary

```
import torch
from torch import nn
image = torch.randn(3, 10, 20)
d0 = image.nelement()

class mynet(nn.Module):
    def __init__(self, d0,d1,d2,d3):
        super().__init__()
        self.m0 = nn.Linear(d0, d1)
        self.m1 = nn.Linear(d1, d2)
        self.m2 = nn.Linear(d2, d3)
    def forward(self, x):
        z0 = x.view(-1)  ## flatten input tensor
        s1 = self.m0(x)
        z1 = torch.relu(s1)
        s2 = self.m1(z1)
        z2 = torch.relu(s2)
        s3 = self.m2(z2)
        return s3

model = mynet(d0,60,40,10)
out = model(image)
```



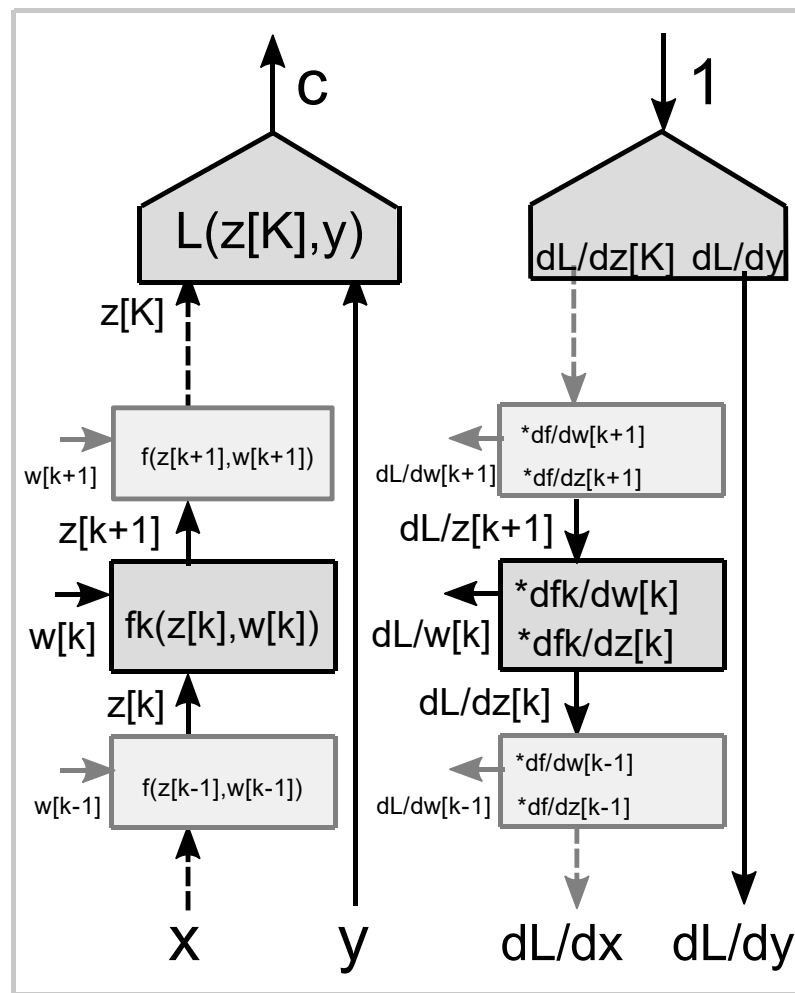
深度学习-反向传播

使用链式法则求导

$$\frac{\partial L}{\partial z_k} = \frac{\partial L}{\partial z_{k+1}} \frac{\partial z_{k+1}}{\partial z_k} = \frac{\partial L}{\partial z_{k+1}} \frac{\partial f(z_k, w_k)}{\partial z_k}$$

$$\frac{\partial L}{\partial w_k} = \frac{\partial L}{\partial z_{k+1}} \frac{\partial z_{k+1}}{\partial w_k} = \frac{\partial L}{\partial z_{k+1}} \frac{\partial f(z_k, w_k)}{\partial w_k}$$

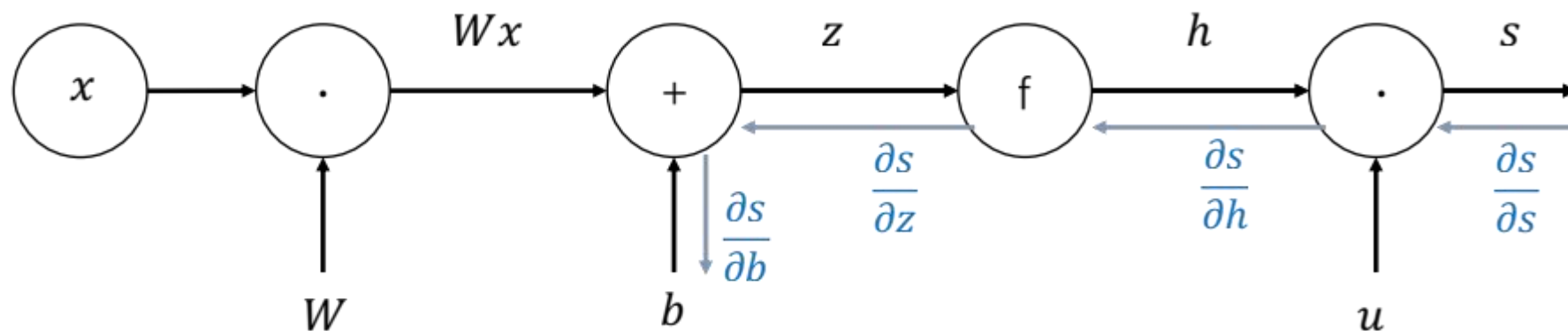
两个**Jacobian**矩阵:
对 $z[k]$ 求导、对 $w[k]$ 求导



深度学习-反向传播

使用链式法则求导

$$\begin{aligned}s &= u^T h \\ h &= f(z) \\ z &= Wx + b \\ x &\text{ input}\end{aligned}$$



$$\frac{\partial L}{\partial z_k} = \frac{\partial L}{\partial z_{k+1}} \frac{\partial z_{k+1}}{\partial z_k} = \frac{\partial L}{\partial z_{k+1}} \frac{\partial f(z_k, w_k)}{\partial z_k}$$

$$\frac{\partial L}{\partial w_k} = \frac{\partial L}{\partial z_{k+1}} \frac{\partial z_{k+1}}{\partial w_k} = \frac{\partial L}{\partial z_{k+1}} \frac{\partial f(z_k, w_k)}{\partial z w_k}$$

深度学习-示例

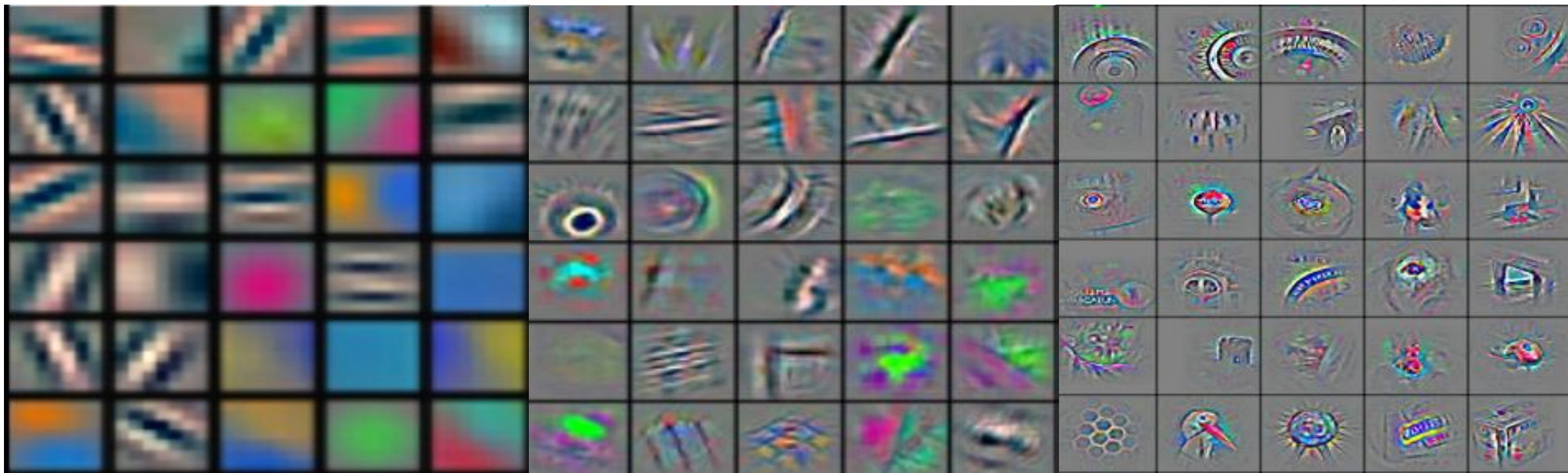


低层次特征

中等层次特征

高层次特征

可训练分类器



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

深度学习-层次表达

图像识别:

Pixel $\rightarrow$ edge $\rightarrow$ texon $\rightarrow$ motif $\rightarrow$ part $\rightarrow$ object

文本

Character $\rightarrow$ word $\rightarrow$ word group $\rightarrow$ clause $\rightarrow$ sentence $\rightarrow$ story

每层都是可学习向量

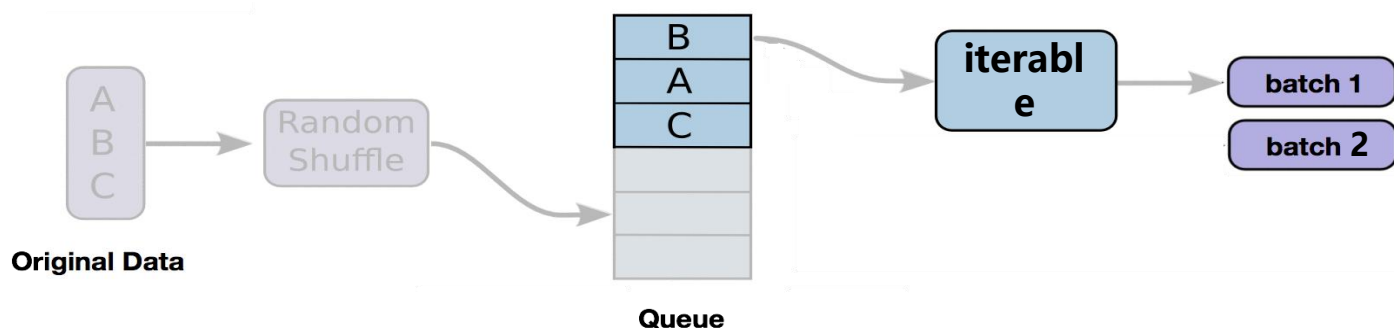
随着层次增加, 信息的抽象表达能力增加

语音

Sample $\rightarrow$ spectral band $\rightarrow$ sound $\rightarrow$... $\rightarrow$ phone $\rightarrow$ phoneme $\rightarrow$ word



深度学习模块化-数据加载器



```

import torch
import numpy as np
from torch.autograd import Variable
from torch.utils.data import Dataset, DataLoader

```

```

class DiabetesDataset(Dataset):
    """ Diabetes dataset. """

    # Initialize your data, download, etc.
    def __init__(self):
        xy = np.loadtxt('data-diabetes.csv', delimiter=',', dtype=np.float32)
        self.len = xy.shape[0]
        self.x_data = torch.from_numpy(xy[:, 0:-1])
        self.y_data = torch.from_numpy(xy[:, [-1]])

    def __getitem__(self, index):
        return self.x_data[index], self.y_data[index]

    def __len__(self):
        return self.len

```

```

dataset = DiabetesDataset()
train_loader = DataLoader(dataset=dataset,
                           batch_size=32,
                           shuffle=True,
                           num_workers=2)

```

```
class Model(torch.nn.Module):
```

```

    def __init__(self):
        """
        In the constructor we instantiate two nn.Linear module
        """
        super(Model, self).__init__()
        self.l1 = torch.nn.Linear(8, 6)
        self.l2 = torch.nn.Linear(6, 4)
        self.l3 = torch.nn.Linear(4, 1)

        self.sigmoid = torch.nn.Sigmoid()

    def forward(self, x):
        """
        In the forward function we accept a Variable of input data and we must return
        a Variable of output data. We can use Modules defined in the constructor as
        well as arbitrary operators on Variables.
        """
        out1 = self.sigmoid(self.l1(x))
        out2 = self.sigmoid(self.l2(out1))
        y_pred = self.sigmoid(self.l3(out2))
        return y_pred

# our model
model = Model()

# Construct our loss function and an Optimizer. The call to model.parameters()
# in the SGD constructor will contain the learnable parameters of the two
# nn.Linear modules which are members of the model.
criterion = torch.nn.BCELoss(size_average=True)
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

# Training loop
for epoch in range(2):
    for i, data in enumerate(train_loader, 0):
        # get the inputs
        inputs, labels = data

        # wrap them in Variable
        inputs, labels = Variable(inputs), Variable(labels)

        # Forward pass: Compute predicted y by passing x to the model
        y_pred = model(inputs)

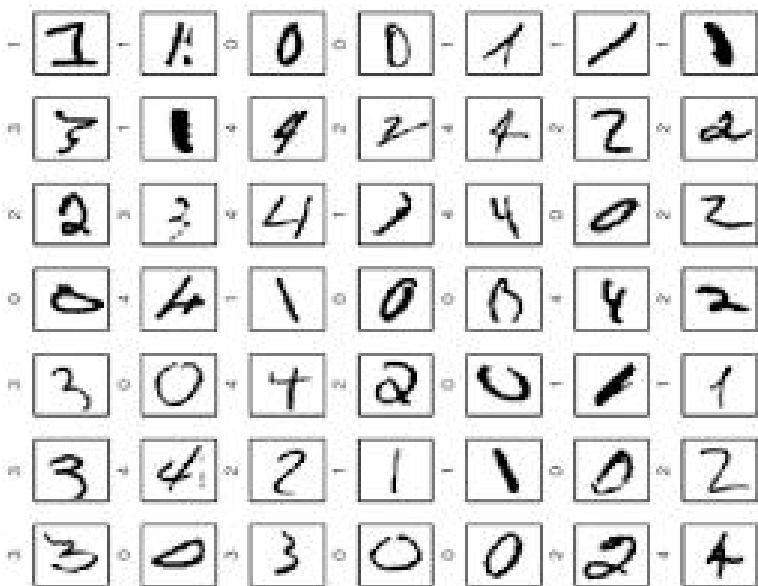
        # Compute and print loss
        loss = criterion(y_pred, labels)
        print(epoch, i, loss.data[0])

        # Zero gradients, perform a backward pass, and update the weights.
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

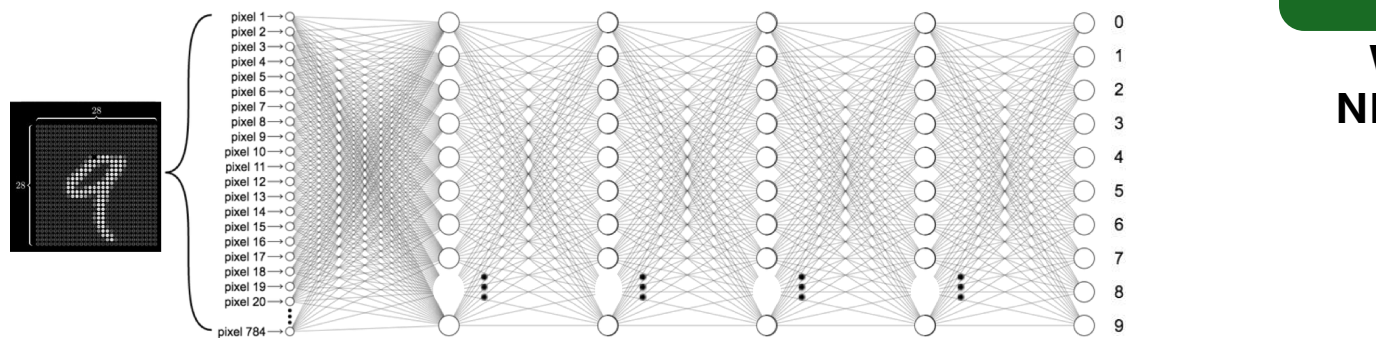
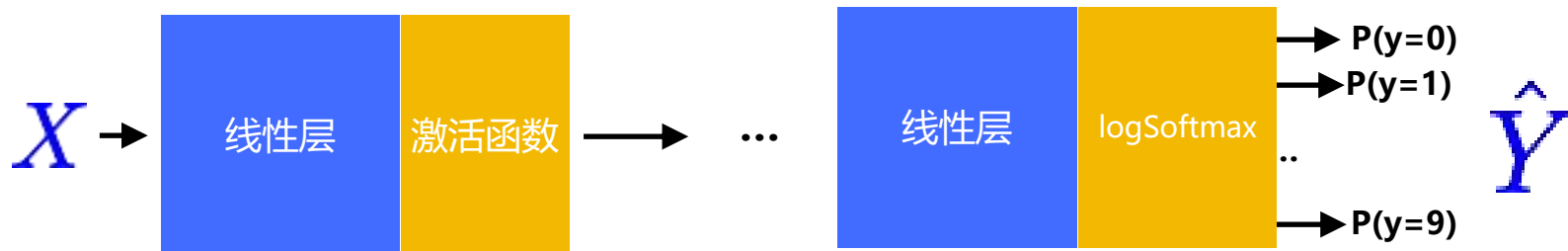
```

图像分类示例

MNIST: 10类



(log)Softmax + NLLLoss



NLLLoss

With
NLLLoss

```
self.l1 = nn.Linear(784, 520)
self.l2 = nn.Linear(520, 320)
self.l3 = nn.Linear(320, 240)
self.l4 = nn.Linear(240, 120)
self.l5 = nn.Linear(120, 10)
```



代码示例

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
```

Training settings

```
batch_size = 64
```

Data loaders

```
train_loader = torch.utils.data.DataLoader(
    datasets.MNIST('./data', train=True, download=True,
        transform=transforms.Compose([
            transforms.ToTensor(),
            transforms.Normalize((0.1307,), (0.3081,))
        ])),
    batch_size=batch_size, shuffle=True)
```

```
test_loader = torch.utils.data.DataLoader(
    datasets.MNIST('./data', train=False,
        transform=transforms.Compose([
            transforms.ToTensor(),
            transforms.Normalize((0.1307,), (0.3081,))
        ])),
    batch_size=batch_size, shuffle=True)
```

Define the Network

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.l1 = nn.Linear(784, 520)
        self.l2 = nn.Linear(520, 320)
        self.l3 = nn.Linear(320, 240)
        self.l4 = nn.Linear(240, 120)
        self.l5 = nn.Linear(120, 10)

    def forward(self, x):
        x = x.view(-1, 784) # Flatten the data (n, 1, 28, 28)-> (n, 784)
        x = F.relu(self.l1(x))
        x = F.relu(self.l2(x))
        x = F.relu(self.l3(x))
        x = F.relu(self.l4(x))
        x = F.relu(self.l5(x))
        return F.log_softmax(x, dim=1) # Added dim=1 for softmax
```

```
model = Net()
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.5)
```

Training

```
def train(epoch):
    model.train()
    for batch_idx, (data, target) in enumerate(train_loader):
        optimizer.zero_grad()
        output = model(data)
        loss = F.nll_loss(output, target)
        loss.backward()
        optimizer.step()
        if batch_idx % 10 == 0:
            print('Train Epoch: {} [{}/{} ({:.0f}%)]\tLoss: {:.6f}'.format(
                epoch, batch_idx * len(data), len(train_loader.dataset),
                100. * batch_idx / len(train_loader), loss.item())) # changed
loss.data[0] to loss.item()
```

evaluation

```
def test():
    model.eval() # Set the model to evaluation mode
    test_loss = 0
    correct = 0
    with torch.no_grad(): # Disable gradient calculation during testing
        for data, target in test_loader:
            output = model(data)
            test_loss += F.nll_loss(output, target, reduction='sum').item() # Sum
up batch loss
            pred = output.argmax(dim=1, keepdim=True) # Get the index of the
max log-probability
            correct += pred.eq(target.view_as(pred)).sum().item()

    test_loss /= len(test_loader.dataset)

    print('\nTest set: Average loss: {:.4f}, Accuracy: {}/{} ({:.0f}%)'.format(
        test_loss, correct, len(test_loader.dataset),
        100. * correct / len(test_loader.dataset)))
```

```
if __name__ == '__main__': # Added to make sure the code runs when
this file is executed directly
```

```
    for epoch in range(1, 11): # Train for 10 epochs
        train(epoch)
```

```
    test() # Evaluate the model after training
```

```
    # Save the Model (Optional)
    torch.save(model.state_dict(), "mnist_model.pth")
    print("Model saved to mnist_model.pth")
```

```
    # Load the Model (Optional - Example of how to load and use the
saved model)
```

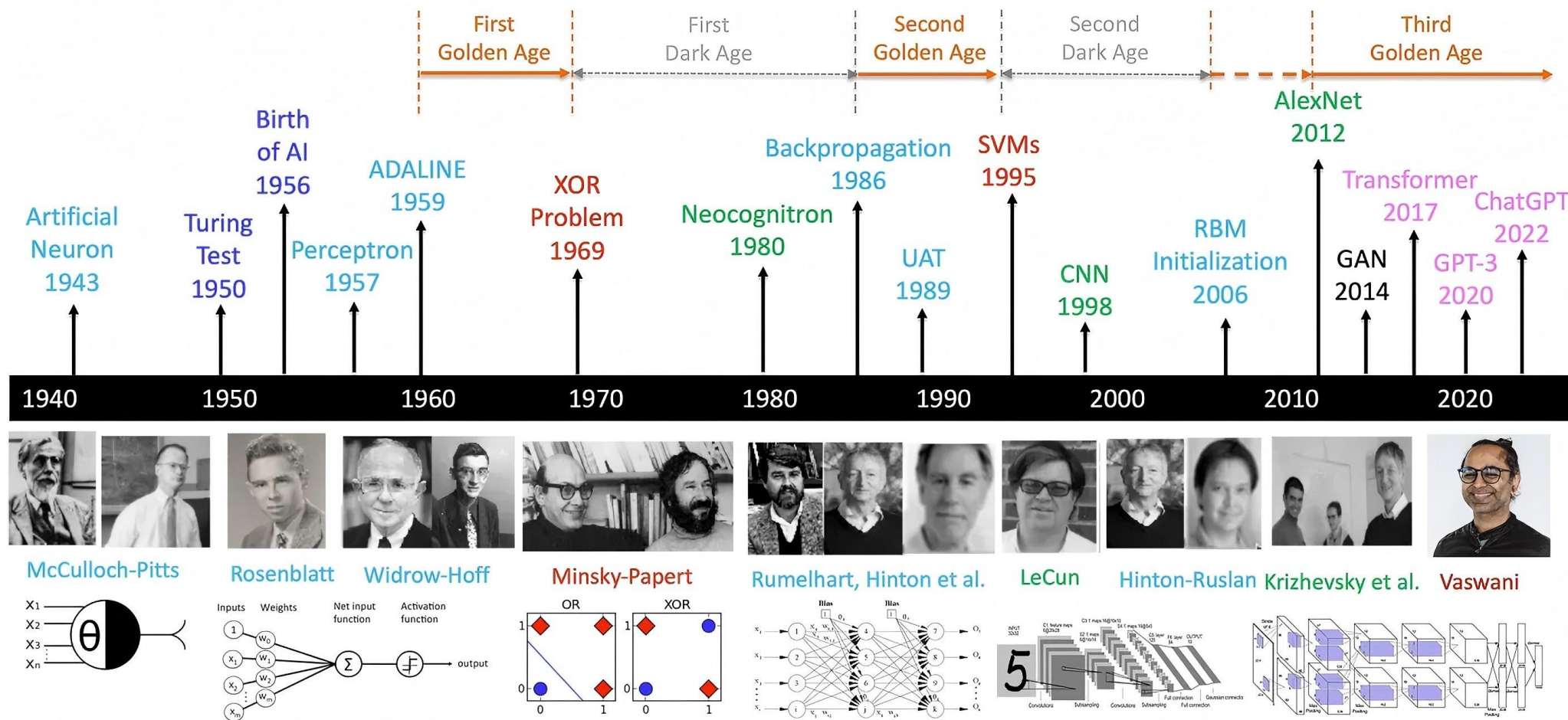
```
    loaded_model = Net()
    loaded_model.load_state_dict(torch.load("mnist_model.pth"))
    loaded_model.eval() # Set to evaluation mode for inference
```

```
    # Example of making a prediction with the loaded model (Optional)
    # Get one batch of test data
    example_data, example_targets = next(iter(test_loader))
    example_output = loaded_model(example_data)
    example_pred = example_output.argmax(dim=1, keepdim=True)
```

```
    print("\nExample Predictions:")
    for i in range(5): # Print predictions for the first 5 examples
        print(f'Example {i+1}: Predicted digit: {example_pred[i].item()},
True digit: {example_targets[i].item()}")
```

深度学习波澜壮阔的历史

A Brief History of AI with Deep Learning



群星闪耀，巨擘辈出，感知器、反向传播等技术横空出世，AlexNet 和 Transformer 等创新力挽狂澜，最终汇聚成今日波澜壮阔的AI浪潮。

人工神经网络诞生 (1943)

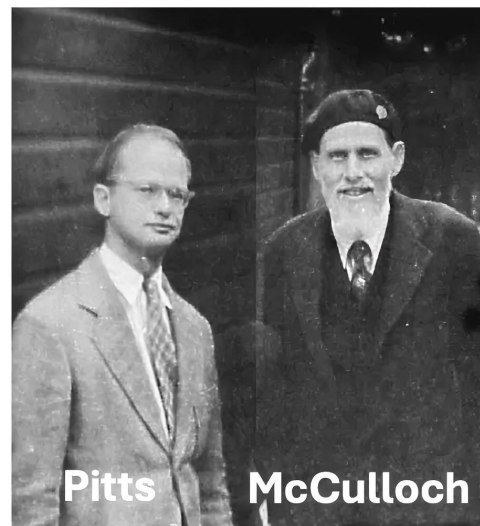
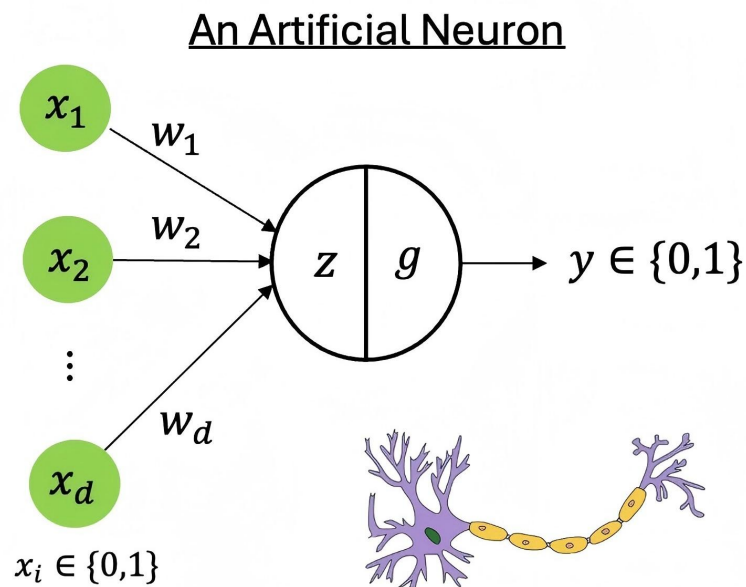
Both inputs and output are binary

- $x_i \in \{0,1\}$ and $y \in \{0,1\}$

$$z(x_1, x_2, \dots, x_d) = z(\mathbf{x}) = \sum_{j=1}^d w_j \cdot x_j$$

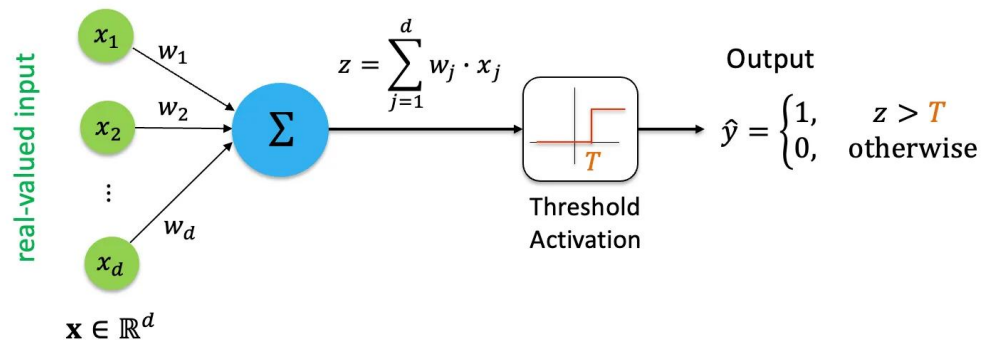
$$y = g(z(\mathbf{x})) = \begin{cases} 1 & \text{if } z(\mathbf{x}) \geq T \\ 0 & \text{if } z(\mathbf{x}) < T \end{cases}$$

Threshold Activation
function



神经网络的概念可以追溯到 1943 年，当时 Warren McCulloch 和 Walter Pitts 提出了第一个人工神经元模型。McCulloch-Pitts (MP) 神经元模型是对生物神经元的开创性简化。该模型通过聚合二元输入并使用阈值激活函数根据此聚合做出决策，从而产生二元输出 $\{0, 1\}$ ，为人工神经网络奠定了基础。

感知器诞生 (1957)



Frank Rosenblatt

$$\mathcal{L}(\mathbf{w}) = - \sum_{i \in \mathcal{M}} \mathbf{w}^T \mathbf{x}_i y_i$$

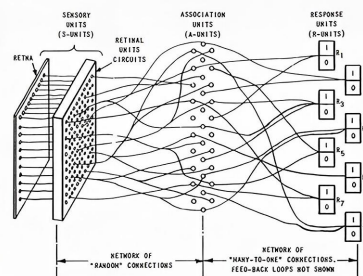
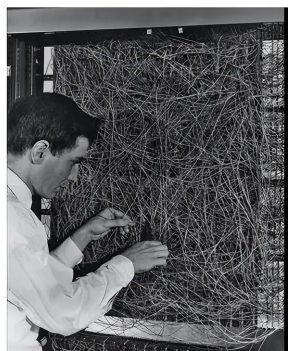


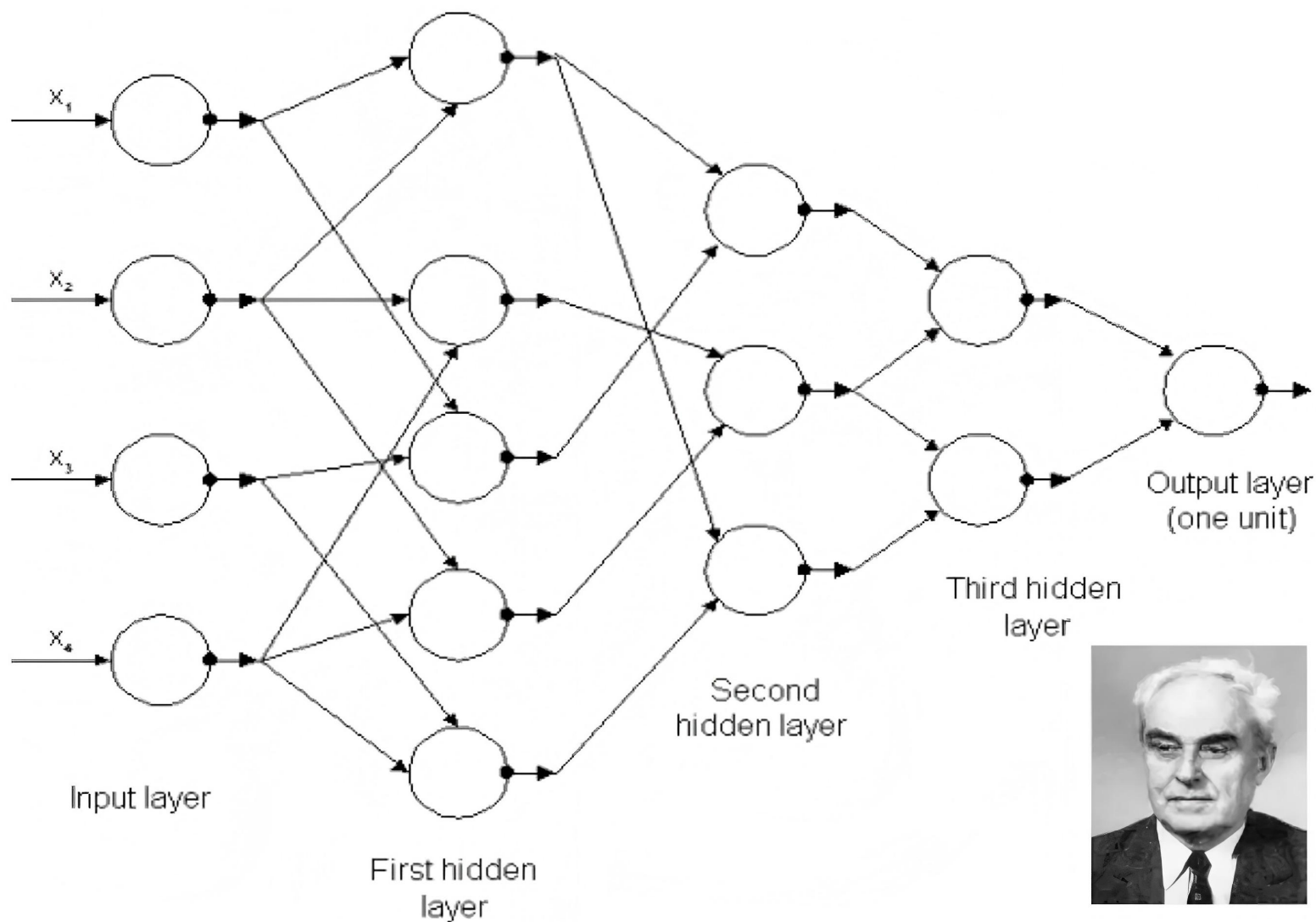
Figure 1 ORGANIZATION OF THE MARK I PERCEPTRON

The Mark I Perceptron was a machine that implemented the perceptron algorithm for image recognition.



1957 年，弗兰克·罗森布拉特 (Frank Rosenblatt) 提出了感知器 (Perceptron)，这是一种能够学习和识别模式的单层神经网络。感知器模型是比 MP 神经元更通用的计算模型，旨在处理实值输入并调整权重以最大限度地减少分类错误。

▶ 多层感知器 (1960s)

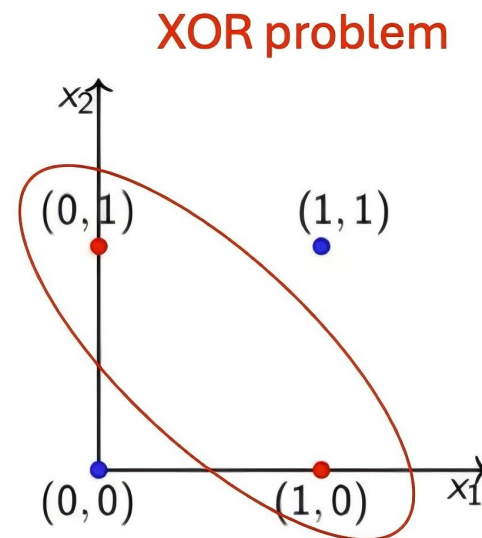
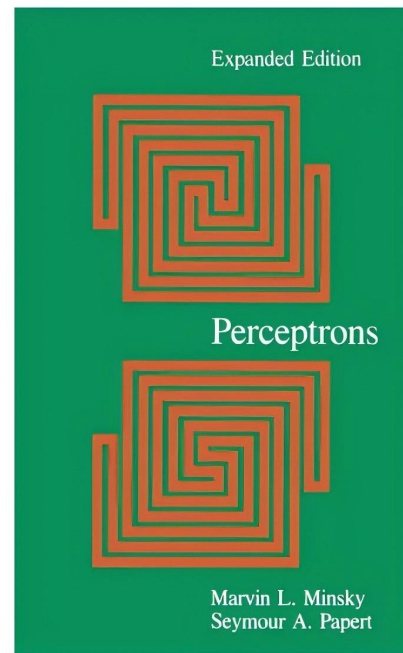


多层感知器 (MLP) 在 1960 年代被引入，作为对单层感知器的改进。它由多层互连的神经元组成，从而能够解决单层模型的局限性。苏联科学家 A. G. 伊瓦赫年科 (A. G. Ivakhnenko) 和 V. 拉帕 (V. Lapa) 在 MLP 的发展中做出了重大贡献，他们在感知器的基础工作之上进行了构建。

▶ XOR问题 (1969)



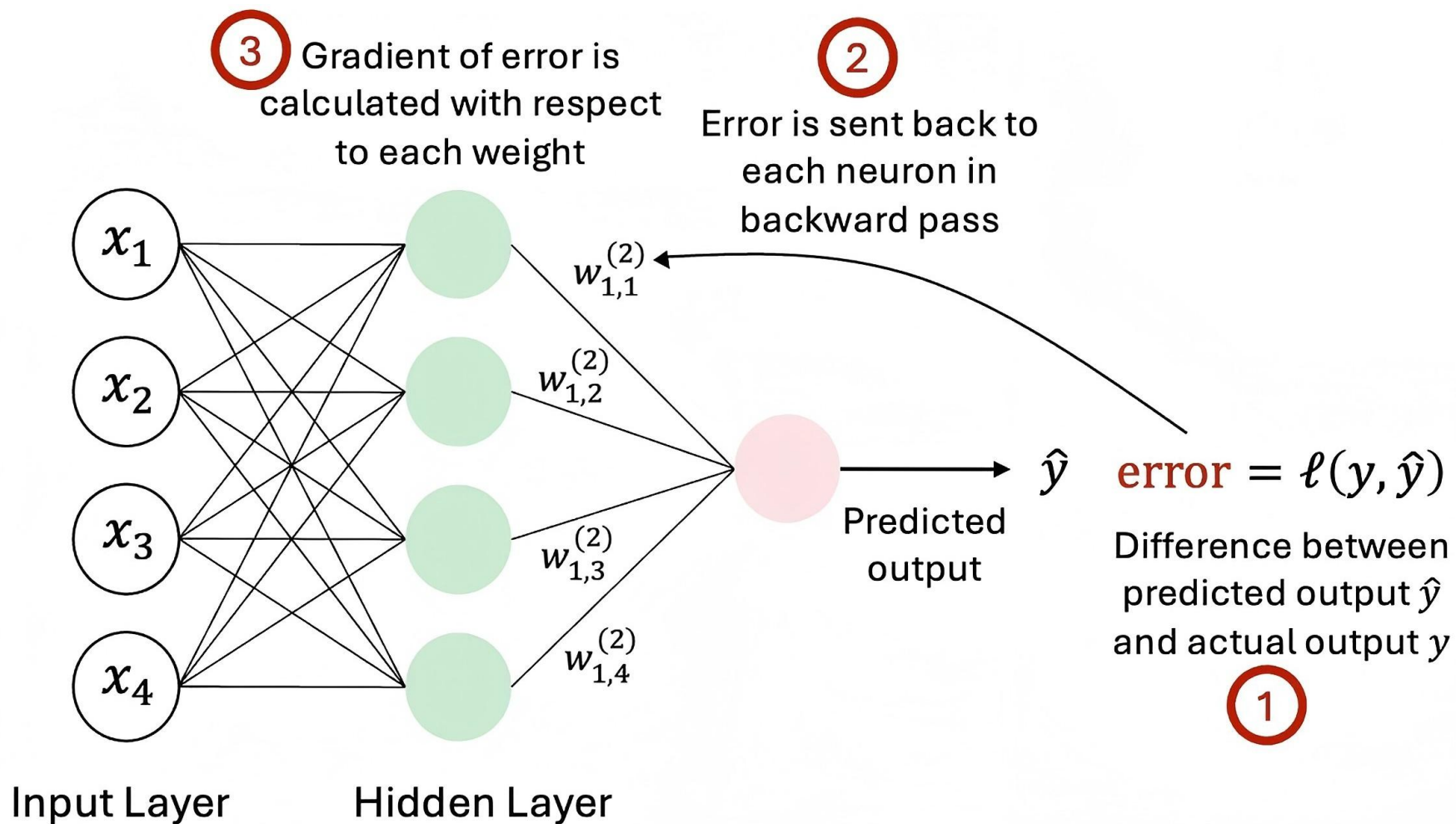
Minsky and Papert: Perceptrons: An introduction to computational geometry. MIT Press, 1969.



1969 年，马文·明斯基 (Marvin Minsky) 和西摩·佩珀特 (Seymour Papert) 在他们的著作《感知器》(Perceptrons) 中强调了单层感知器的一个关键局限性。他们证明，由于其线性决策边界，感知器无法解决异或 (XOR) 问题，这是一个简单的二元分类任务。XOR 问题不是线性可分的，这意味着没有单个线性边界可以正确分类所有输入模式。

这感知器局限性的暴露导致人们对神经网络失去信心，并转向符号人工智能方法，标志着从 1970 年代初期到 1980 年代中期“神经网络的第一个黑暗时代”的开始。

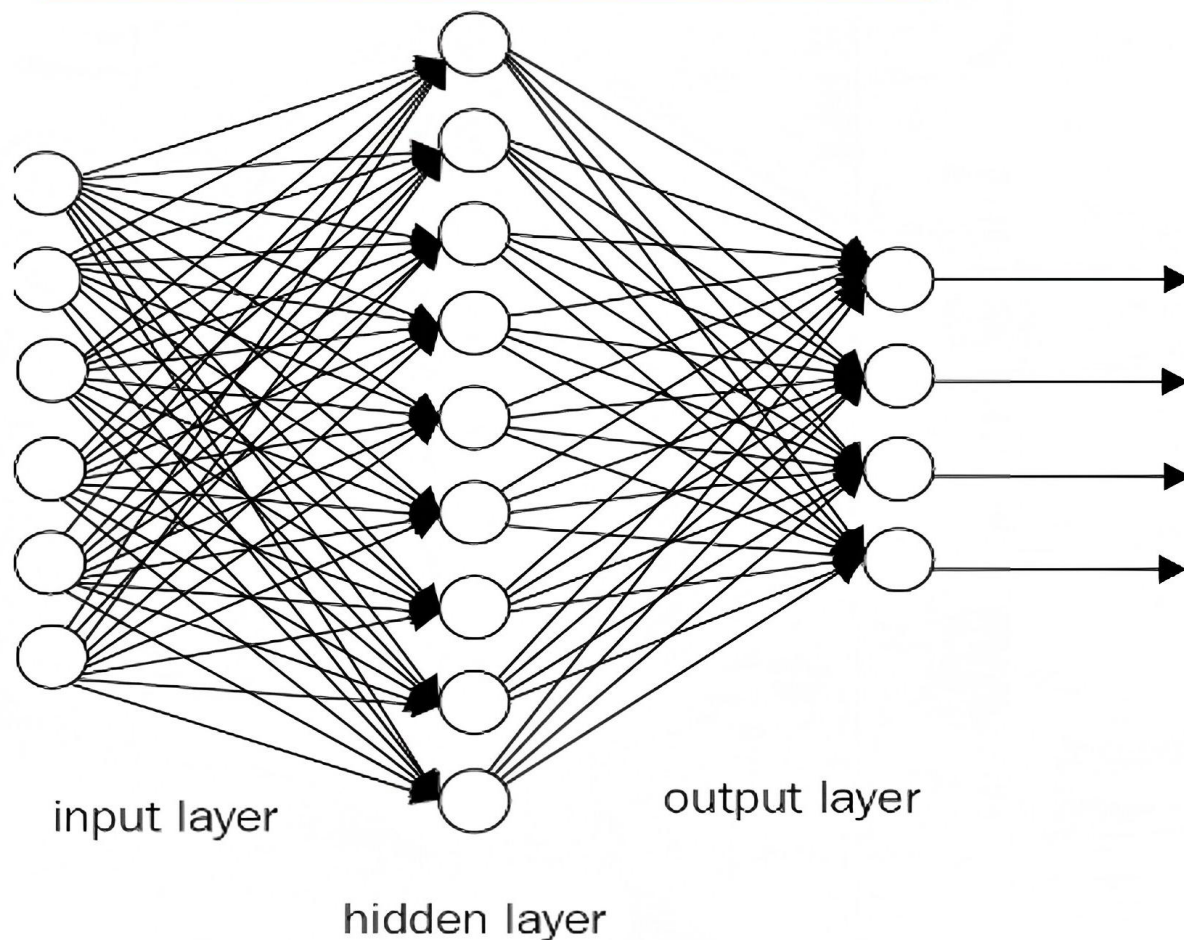
BP算法



(1986 年, David Rumelhart、Geoffrey Hinton 和 Ronald Williams 提出反向传播算法, 将其用于训练深度神经网络。

通用逼近定理

One Hidden Layer is Enough



1989 年，乔治·西本科 (George Cybenko) 提出的通用逼近定理为多层神经网络的能力提供了数学基础。该定理指出，具有单个隐藏层的前馈神经网络，在给定足够多的神经元并使用非线性激活函数的情况下，可以将任何连续函数近似到任意精度。该定理强调了神经网络的强大功能和灵活性，使其适用于广泛的应用。

卷积神经网络崛起



Kenji Fukushima

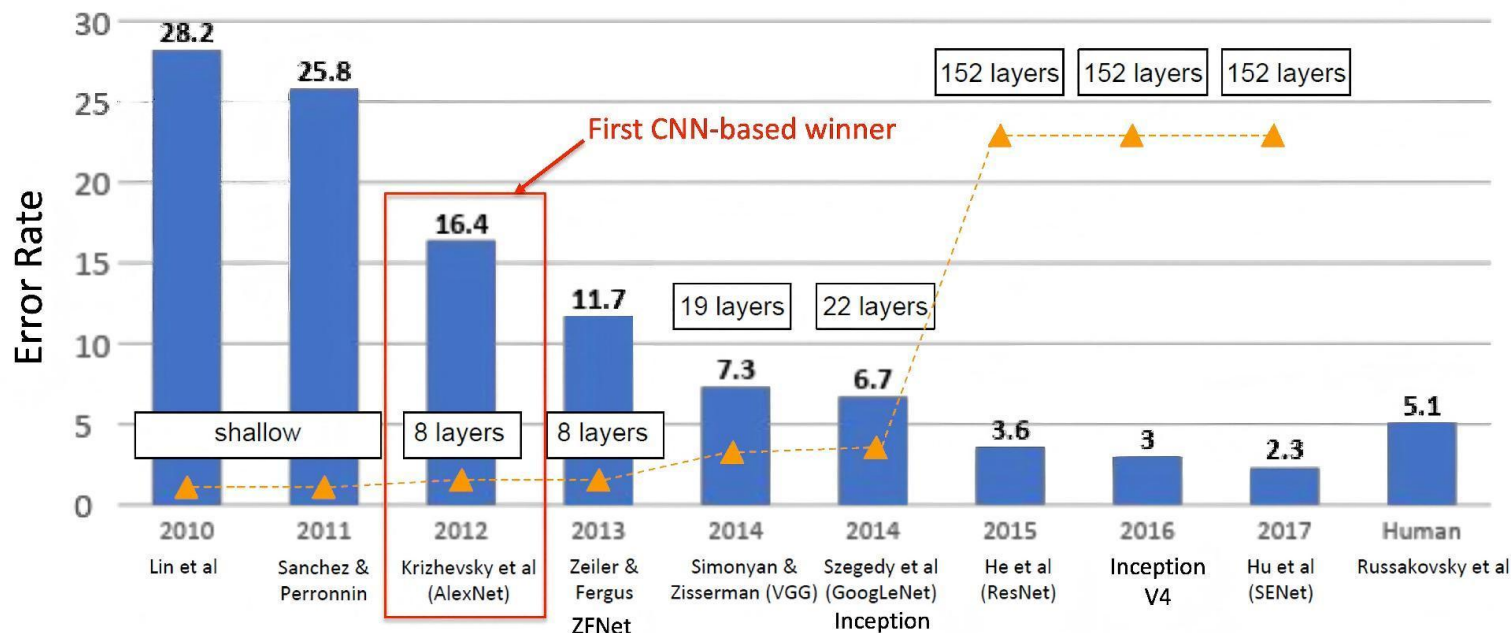
1980年，日本学者福岛邦彦 (Kunihiko Fukushima) 提出神经认知机模型 Neocognitron。



Yann LeCun

1993年，LCun开发了第一个用于文本识别的卷积网络，1998年正式提出了LeNet-5。

Winners of ImageNet Classification Challenge (ILSVRC)



2012年，AlexNet在 ImageNet 图像识别挑战赛中一举夺魁，以 16.4% 的错误率，大幅超越了以往的 shallow 模型，更重要的是，它首次证明了卷积神经网络 (CNN) 在图像识别领域的巨大潜力。AlexNet 的成功，点燃了深度学习在计算机视觉领域的革命之火，开启了 CNN 广泛应用的新时代。

02

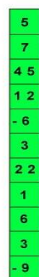
PyTorch

202X

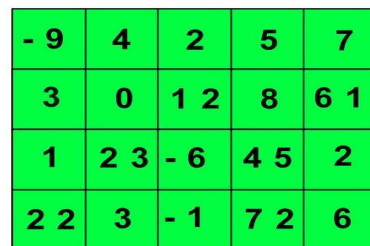
Tensor

- 张量是一个多维数组，可以是标量、向量、矩阵或更高维度的数据结构。
- 在 PyTorch 中，张量 (Tensor) 是数据的核心表示形式，类似于 NumPy 的多维数组，但具有更强大的功能，例如支持 GPU 加速和自动梯度计算。
- 张量支持多种数据类型（整型、浮点型、布尔型等）。
- 张量可以存储在 CPU 或 GPU 中，GPU 张量可显著加速计算。

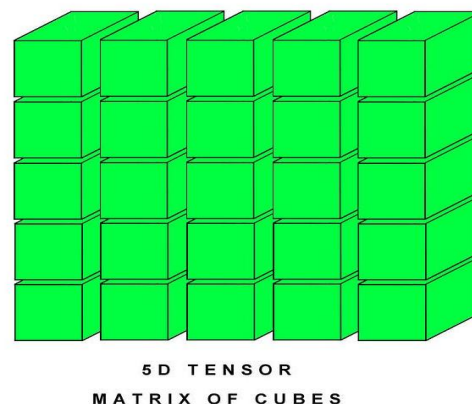
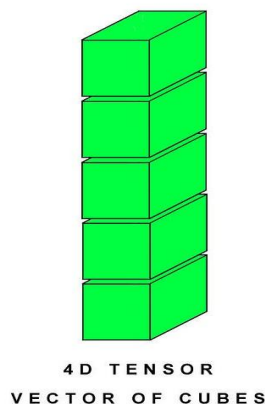
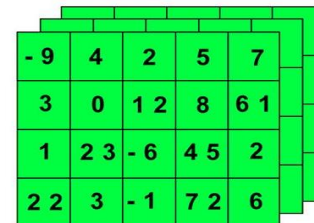
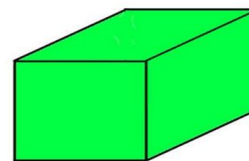
1D TENSOR /
VECTOR



2D TENSOR /
MATRIX



3D TENSOR /
CUBE



- 张量可以存储在 CPU 或 GPU 中，GPU 张量可显著加速计算。
- 1D Tensor / Vector (一维张量/向量)：最基本的张量形式，可以看作是一个数组。
- 2D Tensor / Matrix (二维张量/矩阵)：二维数组，通常用于表示矩阵。
- 3D Tensor / Cube (三维张量/立方体)：三维数组，可以看作是由多个矩阵堆叠而成的立方体
- 4D Tensor / Vector of Cubes (四维张量/立方体向量)：四维数组，可以看作是由多个立方体组成的向量。
- 5D Tensor / Matrix of Cubes (五维张量/立方体矩阵)：五维数组，可以看作是由多个 4D 张量组成的矩阵。

Tensor创建

方法	说明	示例代码
<code>torch.tensor(data)</code>	从 Python 列表或 NumPy 数组创建张量。	<code>x = torch.tensor([[1, 2], [3, 4]])</code>
<code>torch.zeros(size)</code>	创建一个全为零的张量。	<code>x = torch.zeros((2, 3))</code>
<code>torch.ones(size)</code>	创建一个全为 1 的张量。	<code>x = torch.ones((2, 3))</code>
<code>torch.empty(size)</code>	创建一个未初始化的张量。	<code>x = torch.empty((2, 3))</code>
<code>torch.rand(size)</code>	创建一个服从均匀分布的随机张量，值在 $[0, 1)$ 。	<code>x = torch.rand((2, 3))</code>
<code>torch.randn(size)</code>	创建一个服从正态分布的随机张量，均值为 0，标准差为 1。	<code>x = torch.randn((2, 3))</code>
<code>torch.arange(start, end, step)</code>	创建一个一维序列张量，类似于 Python 的 <code>range</code> 。	<code>x = torch.arange(0, 10, 2)</code>
<code>torch.linspace(start, end, steps)</code>	创建一个在指定范围内等间隔的序列张量。	<code>x = torch.linspace(0, 1, 5)</code>
<code>torch.eye(size)</code>	创建一个单位矩阵（对角线为 1，其他为 0）。	<code>x = torch.eye(3)</code>
<code>torch.from_numpy(ndarray)</code>	将 NumPy 数组转换为张量。	<code>x = torch.from_numpy(np.array([1, 2, 3]))</code>

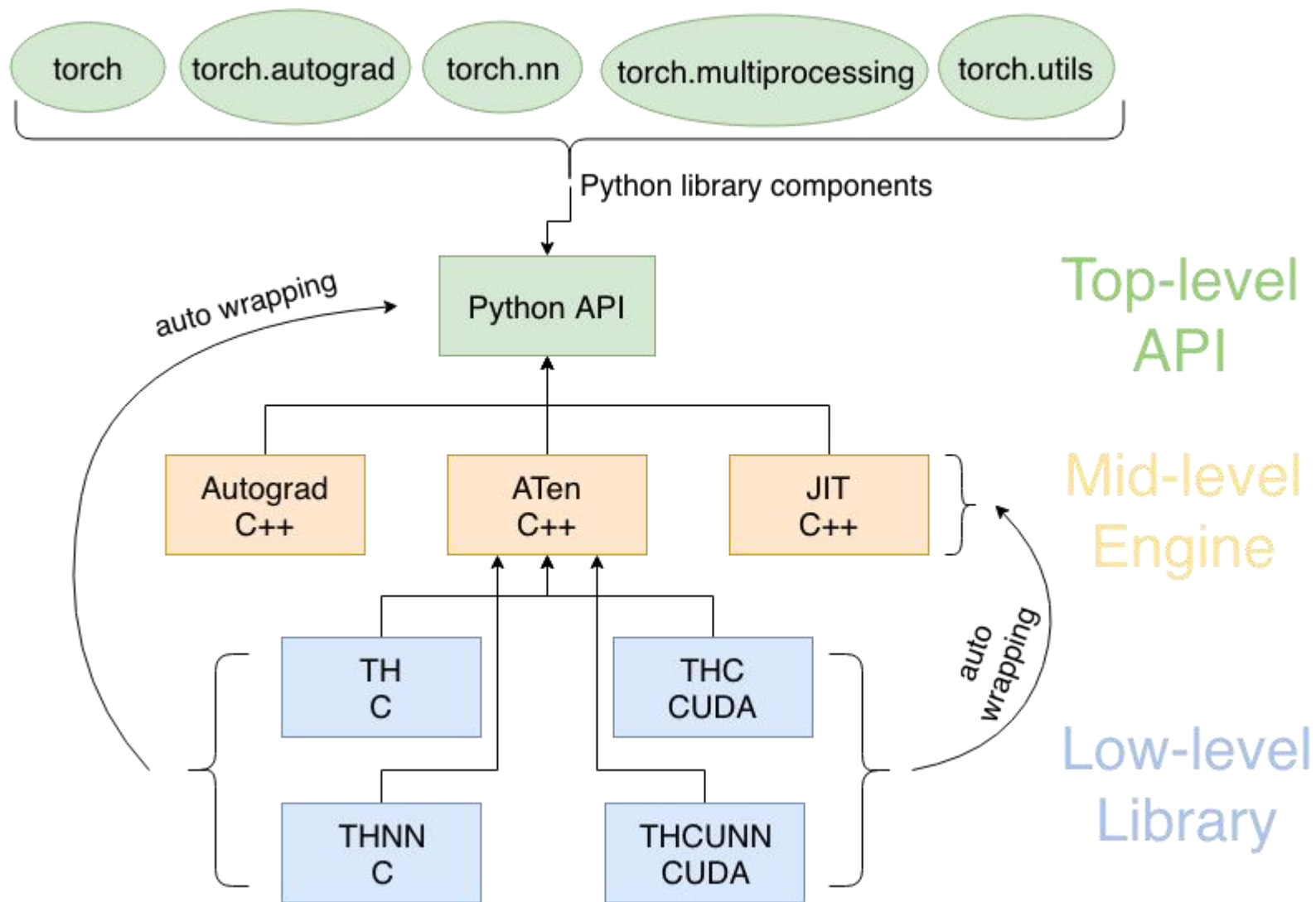
► Tensor属性

属性	说明	示例
<code>.shape</code>	获取张量的形状	<code>tensor.shape</code>
<code>.size()</code>	获取张量的形状	<code>tensor.size()</code>
<code>.dtype</code>	获取张量的数据类型	<code>tensor.dtype</code>
<code>.device</code>	查看张量所在的设备 (CPU/GPU)	<code>tensor.device</code>
<code>.dim()</code>	获取张量的维度数	<code>tensor.dim()</code>
<code>.requires_grad</code>	是否启用梯度计算	<code>tensor.requires_grad</code>
<code>.numel()</code>	获取张量中的元素总数	<code>tensor.numel()</code>
<code>.is_cuda</code>	检查张量是否在 GPU 上	<code>tensor.is_cuda</code>
<code>.T</code>	获取张量的转置 (适用于 2D 张量)	<code>tensor.T</code>
<code>.item()</code>	获取单元素张量的值	<code>tensor.item()</code>
<code>.is_contiguous()</code>	检查张量是否连续存储	<code>tensor.is_contiguous()</code>

Tensor操作

操作	说明	示例代码
<code>+, -, *, /</code>	元素级加法、减法、乘法、除法。	<code>z = x + y</code>
<code>torch.matmul(x, y)</code>	矩阵乘法。	<code>z = torch.matmul(x, y)</code>
<code>torch.dot(x, y)</code>	向量点积（仅适用于 1D 张量）。	<code>z = torch.dot(x, y)</code>
<code>torch.sum(x)</code>	求和。	<code>z = torch.sum(x)</code>
<code>torch.mean(x)</code>	求均值。	<code>z = torch.mean(x)</code>
<code>torch.max(x)</code>	求最大值。	<code>z = torch.max(x)</code>
<code>torch.min(x)</code>	求最小值。	<code>z = torch.min(x)</code>
<code>torch.argmax(x, dim)</code>	返回最大值的索引（指定维度）。	<code>z = torch.argmax(x, dim=1)</code>
<code>torch.softmax(x, dim)</code>	计算 softmax（指定维度）。	<code>z = torch.softmax(x, dim=1)</code>
操作	说明	示例代码
<code>+, -, *, /</code>	元素级加法、减法、乘法、除法。	<code>z = x + y</code>
<code>torch.from_numpy(ndarray)</code>	将 NumPy 数组转换为张量。	<code>x = torch.from_numpy(np_array)</code>
<code>x.numpy()</code>	将张量转换为 NumPy 数组（仅限 CPU 张量）。	<code>np_array = x.numpy()</code>

PyTorch Module Structure Model



PyTorch Codeline Model

- [tools](#) - Code generation scripts for the PyTorch library. See [README](#) of this directory for more details.
- [test](#) - Python unit tests for PyTorch Python frontend.
 - [test_torch.py](#) - Basic tests for PyTorch functionality.
 - [test_autograd.py](#) - Tests for non-NN automatic differentiation support.
 - [test_nn.py](#) - Tests for NN operators and their automatic differentiation.
 - [test_jit.py](#) - Tests for the JIT compiler and TorchScript.
 - ...
 - [cpp](#) - C++ unit tests for PyTorch C++ frontend.
 - [api](#) - [README](#)
 - [jit](#) - [README](#)
 - [tensorexpr](#) - [README](#)
 - [expect](#) - Automatically generated "expect" files which are used to compare against expected output.
 - [onnx](#) - Tests for ONNX export functionality, using both PyTorch and Caffe2.
- [caffe2](#) - The Caffe2 library.
 - [core](#) - Core files of Caffe2, e.g., tensor, workspace, blobs, etc.
 - [operators](#) - Operators of Caffe2.
 - [python](#) - Python bindings to Caffe2.
 - ...
- [.circleci](#) - CircleCI configuration management. [README](#)

- [c10](#) - Core library files that work everywhere, both server and mobile. We are slowly moving pieces from [ATen/core](#) here. This library is intended only to contain essential functionality, and appropriate to use in settings where binary size matters. (But you'll have a lot of missing functionality if you try to use it directly.)
- [aten](#) - C++ tensor library for PyTorch (no autograd support)
 - [src](#) - [README](#)
 - [ATen](#)
 - [core](#) - Core functionality of ATen. This is migrating to top-level c10 folder.
 - [native](#) - Modern implementations of operators. If you want to write a new operator, here is where it should go. Most CPU operators go in the top level directory, except for operators which need to be compiled specially; see [cpu](#) below.
 - [cpu](#) - Not actually CPU implementations of operators, but specifically implementations which are compiled with processor-specific instructions, like AVX. See the [README](#) for more details.
 - [cuda](#) - CUDA implementations of operators.
 - [sparse](#) - CPU and CUDA implementations of COO sparse tensor operations
 - [mkl](#) [mkldnn](#) [miopen](#) [cudnn](#)
 - implementations of operators which simply bind to some backend library.
 - [quantized](#) - Quantized tensor (i.e. QTensor) operation implementations. [README](#) contains details including how to implement native quantized operations.
- [torch](#) - The actual PyTorch library. Everything that is not in [csrc](#) is a Python module, following the PyTorch Python frontend module structure.
 - [csrc](#) - C++ files composing the PyTorch library. Files in this directory tree are a mix of Python binding code, and C++ heavy lifting. Consult [setup.py](#) for the canonical list of Python binding files; conventionally, they are often prefixed with `python_`. [README](#)
 - [jit](#) - Compiler and frontend for TorchScript JIT frontend. [README](#)
 - [autograd](#) - Implementation of reverse-mode automatic differentiation. [README](#)
 - [api](#) - The PyTorch C++ frontend.
 - [distributed](#) - Distributed training support for PyTorch.

<https://github.com/pytorch/pytorch/blob/main/CONTRIBUTING.md#codebase-structure>



深度学习模块化-网络结构 torch.nn

- Containers
- Convolution Layers
- Pooling layers
- Padding Layers
- Non-linear Activations (weighted sum, nonlinearity)
- Non-linear Activations (other)
- Normalization Layers
- Recurrent Layers
- Transformer Layers
- Linear Layers
- Dropout Layers
- Sparse Layers
- Distance Functions
- Loss Functions
- Vision Layers
- Shuffle Layers
- DataParallel Layers (multi-GPU, distributed)
- Utilities
- Quantized Functions
- Lazy Modules Initialization
 - Aliases

Containers

`Module`

Base class for all neural network modules.

`Sequential`

A sequential container.

`ModuleList`

Holds submodules in a list.

`ModuleDict`

Holds submodules in a dictionary.

`ParameterList`

Holds parameters in a list.

`ParameterDict`

Holds parameters in a dictionary.



深度学习模块化-网络结构 torch.nn

- Containers
- Convolution Layers
- Pooling layers
- Padding Layers
- Non-linear Activations (weighted sum, nonlinearity)
- Non-linear Activations (other)
- Normalization Layers
- Recurrent Layers
- Transformer Layers
- Linear Layers
- Dropout Layers
- Sparse Layers
- Distance Functions
- Loss Functions
- Vision Layers
- Shuffle Layers
- DataParallel Layers (multi-GPU, distributed)
- Utilities
- Quantized Functions
- Lazy Modules Initialization
 - Aliases

Convolution Layers

`nn.Conv1d`

Applies a 1D convolution over an input signal composed of several input planes.

`nn.Conv2d`

Applies a 2D convolution over an input signal composed of several input planes.

`nn.Conv3d`

Applies a 3D convolution over an input signal composed of several input planes.

`nn.ConvTranspose1d`

Applies a 1D transposed convolution operator image composed of several input planes.

`nn.ConvTranspose2d`

Applies a 2D transposed convolution operator image composed of several input planes.

`nn.ConvTranspose3d`

Applies a 3D transposed convolution operator image composed of several input planes.

`nn.LazyConv1d`

A `torch.nn.Conv1d` module with lazy initialize `in_channels` argument.

`nn.LazyConv2d`

A `torch.nn.Conv2d` module with lazy initialize `in_channels` argument.

`nn.LazyConv2d`

A `torch.nn.Conv2d` module with lazy initialization of the `in_channels` argument.

`nn.LazyConv3d`

A `torch.nn.Conv3d` module with lazy initialization of the `in_channels` argument.

`nn.LazyConvTranspose1d`

A `torch.nn.ConvTranspose1d` module with lazy initialization of the `in_channels` argument.

`nn.LazyConvTranspose2d`

A `torch.nn.ConvTranspose2d` module with lazy initialization of the `in_channels` argument.

`nn.LazyConvTranspose3d`

A `torch.nn.ConvTranspose3d` module with lazy initialization of the `in_channels` argument.

`nn.Unfold`

Extracts sliding local blocks from a batched input tensor.

`nn.Fold`

Combines an array of sliding local blocks into a large containing tensor.



深度学习模块化-网络结构 torch.nn

- Containers
- Convolution Layers
- Pooling layers
- Padding Layers
- Non-linear Activations (weighted sum, nonlinearity)
- Non-linear Activations (other)
- Normalization Layers
- Recurrent Layers
- Transformer Layers
- Linear Layers
- Dropout Layers
- Sparse Layers
- Distance Functions
- Loss Functions
- Vision Layers
- Shuffle Layers
- DataParallel Layers (multi-GPU, distributed)
- Utilities
- Quantized Functions
- Lazy Modules Initialization
 - Aliases

Non-linear Activations (weighted sum, nonlinearity)

<code>nn.ELU</code>	Applies the Exponential Linear Unit (ELU) function, element-wise.
<code>nn.Hardshrink</code>	Applies the Hard Shrinkage (Hardshrink) function element-wise.
<code>nn.Hardsigmoid</code>	Applies the Hardsigmoid function element-wise.
<code>nn.Hardtanh</code>	Applies the HardTanh function element-wise.
<code>nn.Hardswish</code>	Applies the Hardswish function, element-wise.
<code>nn.LeakyReLU</code>	Applies the LeakyReLU function element-wise.
<code>nn.LogSigmoid</code>	Applies the Logsigmoid function element-wise.
<code>nn.MultiheadAttention</code>	Allows the model to jointly attend to information from different representation subspaces.
<code>nn.PReLU</code>	Applies the element-wise PReLU function.
<code>nn.ReLU</code>	Applies the rectified linear unit function element-wise.
<code>nn.ReLU6</code>	Applies the ReLU6 function element-wise.
<code>nn.RReLU</code>	Applies the randomized leaky rectified linear unit function, element-wise.

<code>nn.SELU</code>	Applies the SELU function element-wise.
<code>nn.CELU</code>	Applies the CELU function element-wise.
<code>nn.GELU</code>	Applies the Gaussian Error Linear Units function.
<code>nn.Sigmoid</code>	Applies the Sigmoid function element-wise.
<code>nn.SiLU</code>	Applies the Sigmoid Linear Unit (SiLU) function, element-wise.
<code>nn.Mish</code>	Applies the Mish function, element-wise.
<code>nn.Softplus</code>	Applies the Softplus function element-wise.
<code>nn.Softshrink</code>	Applies the soft shrinkage function element-wise.
<code>nn.Softsign</code>	Applies the element-wise Softsign function.
<code>nn.Tanh</code>	Applies the Hyperbolic Tangent (Tanh) function element-wise.
<code>nn.Tanhshrink</code>	Applies the element-wise Tanhshrink function.
<code>nn.Threshold</code>	Thresholds each element of the input Tensor.
<code>nn.GLU</code>	Applies the gated linear unit function.

深度学习模块化-网络结构 torch.nn

- Containers
- Convolution Layers
- Pooling layers
- Padding Layers
- Non-linear Activations (weighted sum, nonlinearity)
- Non-linear Activations (other)
- Normalization Layers
- Recurrent Layers
- Transformer Layers
- Linear Layers
- Dropout Layers
- Sparse Layers
- Distance Functions
- Loss Functions
- Vision Layers
- Shuffle Layers
- DataParallel Layers (multi-GPU, distributed)
- Utilities
- Quantized Functions
- Lazy Modules Initialization
 - Aliases

Activation function	Equation	Example	1D Graph
Unit step (Heaviside)	$\phi(z) = \begin{cases} 0, & z < 0, \\ 0.5, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Sign (Signum)	$\phi(z) = \begin{cases} -1, & z < 0, \\ 0, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Linear	$\phi(z) = z$	Adaline, linear regression	
Piece-wise linear	$\phi(z) = \begin{cases} 1, & z \geq \frac{1}{2}, \\ z + \frac{1}{2}, & -\frac{1}{2} < z < \frac{1}{2}, \\ 0, & z \leq -\frac{1}{2}, \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multi-layer NN	
Hyperbolic tangent	$\phi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	Multi-layer Neural Networks	
Rectifier, ReLU (Rectified Linear Unit)	$\phi(z) = \max(0, z)$	Multi-layer Neural Networks	
Rectifier, softplus	$\phi(z) = \ln(1 + e^z)$	Multi-layer Neural Networks	

Copyright © Sebastian Raschka 2016
(<http://sebastianraschka.com>)

http://rasbt.github.io/mlxtend/user_guide/general_concepts/activation-functions/



深度学习模块化-网络结构 torch.nn

- Containers
- Convolution Layers
- Pooling layers
- Padding Layers
- Non-linear Activations (weighted sum, nonlinearity)
- Non-linear Activations (other)
- Normalization Layers
- Recurrent Layers
- Transformer Layers
- Linear Layers
- Dropout Layers
- Sparse Layers
- Distance Functions
- Loss Functions
- Vision Layers
- Shuffle Layers
- DataParallel Layers (multi-GPU, distributed)
- Utilities
- Quantized Functions
- Lazy Modules Initialization
 - Aliases

Transformer Layers [🔗](#)

`nn.Transformer`

A transformer model.

`nn.TransformerEncoder`

TransformerEncoder is a stack of N encoder layers.

`nn.TransformerDecoder`

TransformerDecoder is a stack of N decoder layers.

`nn.TransformerEncoderLayer`

TransformerEncoderLayer is made up of self-attn and feedforward network.

`nn.TransformerDecoderLayer`

TransformerDecoderLayer is made up of self-attn, multi-head-attn and feedforward network.



深度学习模块化-网络结构 torch.nn

- Containers
- Convolution Layers
- Pooling layers
- Padding Layers
- Non-linear Activations (weighted sum, nonlinearity)
- Non-linear Activations (other)
- Normalization Layers
- Recurrent Layers
- Transformer Layers
- Linear Layers
- Dropout Layers
- Sparse Layers
- Distance Functions
- Loss Functions
- Vision Layers
- Shuffle Layers
- DataParallel Layers (multi-GPU, distributed)
- Utilities
- Quantized Functions
- Lazy Modules Initialization
 - Aliases

Dropout Layers

`nn.Dropout`

During training, randomly zeroes some of the elements of the input tensor with probability `p`.

`nn.Dropout1d`

Randomly zero out entire channels.

`nn.Dropout2d`

Randomly zero out entire channels.

`nn.Dropout3d`

Randomly zero out entire channels.

`nn.AlphaDropout`

Applies Alpha Dropout over the input.

`nn.FeatureAlphaDropout`

Randomly masks out entire channels.



深度学习模块化-损失函数

Loss functions
L1Loss
MSELoss
CrossEntropyLoss
NLLLoss
PoissonNLLLoss
NLLLoss2d
KLDivLoss
BCELoss
BCEWithLogitsLoss
MarginRankingLoss
HingeEmbeddingLoss
MultiLabelMarginLoss
SmoothL1Loss
SoftMarginLoss
MultiLabelSoftMarginLoss
CosineEmbeddingLoss
MultiMarginLoss
TripletMarginLoss

<http://pytorch.org/docs/master/nn.html>

Table 1: List of losses analysed in this paper. $\mathbf{y}$ is true label as one-hot encoding, $\hat{\mathbf{y}}$ is true label as +1/-1 encoding, $\mathbf{o}$ is the output of the last layer of the network, $\cdot^{(j)}$ denotes j th dimension of a given vector, and $\sigma(\cdot)$ denotes probability estimate.

symbol	name	equation
$\mathcal{L}_1$	L_1 loss	$\ \mathbf{y} - \mathbf{o}\ _1$
$\mathcal{L}_2$	L_2 loss	$\ \mathbf{y} - \mathbf{o}\ _2^2$
$\mathcal{L}_1 \circ \sigma$	expectation loss	$\ \mathbf{y} - \sigma(\mathbf{o})\ _1$
$\mathcal{L}_2 \circ \sigma$	regularised expectation loss ¹	$\ \mathbf{y} - \sigma(\mathbf{o})\ _2^2$
$\mathcal{L}_\infty \circ \sigma$	Chebyshev loss	$\max_j \sigma(\mathbf{o})^{(j)} - \mathbf{y}^{(j)} $
hinge	hinge [13] (margin) loss	$\sum_j \max(0, \frac{1}{2} - \hat{\mathbf{y}}^{(j)} \mathbf{o}^{(j)})$
hinge ²	squared hinge (margin) loss	$\sum_j \max(0, \frac{1}{2} - \hat{\mathbf{y}}^{(j)} \mathbf{o}^{(j)})^2$
hinge ³	cubed hinge (margin) loss	$\sum_j \max(0, \frac{1}{2} - \hat{\mathbf{y}}^{(j)} \mathbf{o}^{(j)})^3$
log	log (cross entropy) loss	$-\sum_j \mathbf{y}^{(j)} \log \sigma(\mathbf{o})^{(j)}$
log ²	squared log loss	$-\sum_j [\mathbf{y}^{(j)} \log \sigma(\mathbf{o})^{(j)}]^2$
tan	Tanimoto loss	$\frac{-\sum_j \sigma(\mathbf{o})^{(j)} \mathbf{y}^{(j)}}{\ \sigma(\mathbf{o})\ _2^2 + \ \mathbf{y}\ _2^2 - \sum_j \sigma(\mathbf{o})^{(j)} \mathbf{y}^{(j)}}$
D _{CS}	Cauchy-Schwarz Divergence [3]	$-\log \frac{\sum_j \sigma(\mathbf{o})^{(j)} \mathbf{y}^{(j)}}{\ \sigma(\mathbf{o})\ _2 \ \mathbf{y}\ _2}$

<https://arxiv.org/pdf/1702.05659.pdf>

2025

谢谢大家

时间：202X.X